Lab 1: Project Foundations and MLflow Connectivity

- **Course:** Engineering of Intelligent Models
- **Module:** M1 – MLOps Foundations
- **Focus Tool:** MLflow
- **Format:** Hands-on Laboratory
- **Branch:** `Lab1`

1. Goal of the Lab

The objective of this session is to establish a professional, scalable project structure and a containerized infrastructure using **Docker Compose**. You will learn how to:

1. Define a standardized MLOps folder hierarchy.
2. Deploy a local **MLflow Tracking Server**.
3. Verify connectivity by logging your first metadata and artifacts from a local CSV dataset.
4. Establish a robust version control strategy using **GitHub** branches.

2. Version Control Strategy

Before writing any code, you must establish your workspace. You have two options for following this course:

- **Independent Project:** Create a new repository in your personal GitHub account named `EMI-WeatherForecast`. This is the recommended approach for your portfolio.
- **Follow the Professor:** Clone the official public repository provided in the course platform.

Once your repository is ready, create your first development branch. This will isolate your work for this specific laboratory.

In [ ]:
```
!git checkout -b Lab1
```

3. Project Folder Structure Configuration

A professional MLOps project requires a predictable directory layout to ensure that orchestration tools like **Airflow** and versioning tools like **DVC** can find the necessary files without manual configuration. The following structure is just an example. You can adapt it as needed, but the key is to maintain consistency across all labs.

```
EMI-WeatherForecast/
├── data/                   # To be managed by DVC
│   └── raw/                # Initial weather.csv goes here
├── dags/                   # Apache Airflow Directed Acyclic Graphs
(DAGs)
├── models/                 # Model artifacts managed by MLflow
├── notebooks/              # EDA, tests, run-ins, etc.
├── src/                    # Source code
│   ├── ingestion/          # API calls (Open-Meteo/Meteostat)
│   └── training/           # Training scripts
│   └── serving/            $ Future home for FastAPI scripts.
├── conf/                   # Hydra configurations
├── docker-compose.yaml     # Infrastructure definition
└── requirements.txt        # Python dependencies
```

Execute the following commands to initialize the project:

> **ATTENTION:** If you are following the professor's repository, this structure is already set up for you. You can skip the folder creation step.

In [ ]:
```
import os

# Determine the project root directory (assuming this notebook is enunciados/Lab1/)
root_dir = os.path.dirname(os.path.dirname(os.getcwd()))
print(f"Project Root Directory: {root_dir}")

folders = ["data/raw", "dags", "models", "notebooks", "src/ingestion", "src/training", "src

for folder in folders:
        path = os.path.join(root_dir, folder)
        os.makedirs(path, exist_ok=True)
        print(f"Created folder: {path}")
```

Change into the project root directory to ensure all commands run in the correct context:

In [1]:
```
import os
root_dir = os.path.dirname(os.path.dirname(os.getcwd()))
os.chdir(root_dir) # Move to the project root directory
```

4. Infrastructure: `docker-compose.yaml`

We will use Docker Compose to manage our services. Currently, we only require **MLflow**, but we will add **Airflow** and other tools to this file in later labs.

Create a file named `docker-compose.yaml` in your root folder:

```
name: emi-weather-forecasting
services:
  mlflow_server:
    image: ghcr.io/mlflow/mlflow:latest
    container_name: emi-mlflow
    ports:
      - "5000:5000"
    environment:
      - MLFLOW_BACKEND_STORE_URI=sqlite:////mlflow/mlflow.db
      - MLFLOW_ARTIFACT_ROOT=/mlflow/artifacts
    volumes:
      - ./mlflow_data:/mlflow
    command: >
      mlflow server
        --backend-store-uri sqlite:////mlflow/mlflow.db
        --default-artifact-root mlflow-artifacts:/
        --artifacts-destination /mlflow/artifacts
        --serve-artifacts
        --host 0.0.0.0
```

Code Explanation:

- `name: emi-weather-forecasting` : This is the name of your Docker Compose project. It will be used as a prefix for container names and network.
- `services` : This section defines the services that will be part of your infrastructure.
- `mlflow_server` : This is the name of the service. You can choose any name you like, but it should be descriptive.

- `image: ghcr.io/mlflow/mlflow:latest` : This specifies the Docker image to use for the MLflow server. We are using the latest version from the GitHub Container Registry.
- `container_name: emi-mlflow` : This sets a custom name for the container. It makes it easier to identify when you run `docker ps` .
- `ports` : This maps port 5000 of the container to port 5000 on your local machine, allowing you to access the MLflow UI via `http://localhost:5000` .
- `environment` : This section sets environment variables for the MLflow server. We specify the backend store URI (using SQLite) and the artifact root directory.
- `volumes` : This mounts a local directory (`./mlflow_data`) to the container's `/mlflow` directory, ensuring that your MLflow data persists even if the container is stopped or removed.
- `command:` : This overrides the default command of the MLflow image to start the server with the specified backend store and artifact root.

Launch the infrastructure

In [3]:
```
!docker-compose up -d
```

```
Container emi-mlflow Running
```

Check the UI at `http://localhost:5000` . You should see the default MLflow dashboard.

(Optional) Create a `.gitignore` file

To prevent local artifacts and logs from being committed to your Git repository, create a `.gitignore` file in the root directory (if you don't have one already) and add the following lines to ignore the MLflow data directory:

```
# Ignore MLflow data and artifacts
mlflow_data/
```

5. Data Setup and Dependencies

Create `requirements.txt` in the root folder to manage your Python environment:

```
mlflow==3.10.0
pandas==2.3.3
```

Install the dependencies:

In [ ]:
```
!pip install -r requirements.txt
```

Create a sample dataset at `data/raw/weather.csv` . This represents an example data we will later pull from APIs like **Open-Meteo**:

```
date,temperature,humidity,pressure
2026-02-01,15.5,60.0,1012.0
2026-02-02,16.2,58.0,1011.0
2026-02-03,14.8,65.0,1014.0
2026-02-04,17.0,55.0,1010.0
```

6. Connecting to MLflow: Experimental Code

Now, write the script that connects your training logic to the "flight recorder".

Create `src/training/mlflow_connection.py` :

```python
import mlflow
from mlflow.data.filesystem_dataset_source import FileSystemDatasetSource
from mlflow.data.pandas_dataset import from_pandas
import pandas as pd
import os

# Set connection to the local server running in Docker
mlflow.set_tracking_uri("http://localhost:5000")
mlflow.set_experiment("Lab1_Connectivity_Test")


def run_test():
    # Load the sample weather data
    data_path = os.path.abspath("data/raw/weather.csv")
    df = pd.read_csv(data_path)

    # Create a Dataset object
    fs_dataset_source = FileSystemDatasetSource()
    dataset = from_pandas(df, source=fs_dataset_source.load(data_path),
name="weather-sample", targets="temperature")

    with mlflow.start_run(run_name="Initial_CSV_Load"):
        # Log parameters (The "Intent")
        mlflow.log_param("df_shape", df.shape)
        mlflow.log_param("row_count", len(df))

        # Log metrics (The measurable results)
        mlflow.log_metric("average_temp", df["temperature"].mean())

        # Log the dataset (The file itself for total traceability)
        mlflow.log_input(dataset, context="training")  # as an input to the
training process
        mlflow.log_artifact(data_path, artifact_path="raw_data")  # as a file
artifact for reference


if __name__ == "__main__":
    run_test()
```

Code Explanation:

- `mlflow.set_tracking_uri("http://localhost:5000")` : This tells your script to connect to the MLflow server running in Docker.
- `mlflow.set_experiment("Lab1_Connectivity_Test")` : This creates a new experiment in MLflow where all runs from this script will be logged.
- `data_path = os.path.abspath("data/raw/weather.csv")` : This ensures that the script can find the dataset regardless of where it is executed from, as long as the relative path is correct.
- `df = pd.read_csv(data_path)` : This loads the CSV data into a Pandas DataFrame.
- `fs_dataset_source = FileSystemDatasetSource()` : This creates a dataset source that can read from the filesystem, which is necessary for MLflow to track the dataset as an artifact.
- `dataset = from_pandas(...)` : This converts the DataFrame into an MLflow Dataset object, which can be logged as an artifact. This is crucial for traceability, as it allows you to see exactly which data was used for this run. It accepts other objects such as **Hugging Face**, **Spark DataFrames**, **Cloud Providers** (e.g., Azurem S3, GCS), and more, making it a versatile option for different data sources.
- `with mlflow.start_run(run_name="Initial_CSV_Load"):` This block starts a new MLflow run, which is a single execution of your training script. Inside this block, you log parameters (like the shape of the DataFrame), metrics (like the average temperature), and the dataset itself as an

artifact. This ensures that every aspect of the experiment is recorded in MLflow, providing complete traceability from the raw data to the logged results.

- `mlflow.log_param("df_shape", df.shape)` : Logs the shape of the DataFrame as a parameter.
- `mlflow.log_param("row_count", len(df))` : Logs the number of rows in the DataFrame as a parameter.
- `mlflow.log_metric("average_temp", df["temperature"].mean())` : Logs the average temperature as a metric.
- `mlflow.log_input(dataset, context="training")` : Logs the dataset itself as an artifact under the "training" context, ensuring that you can trace back exactly which data was used for this run.
- `mlflow.log_artifact(data_path, artifact_path="raw_data")` : Additionally logs the original CSV file as an artifact for reference, allowing you to download it from the MLflow UI if needed.

Run the experiment:

In [21]:
```
!python src/training/mlflow_connection.py
```

```
🏃 View run Initial_CSV_Load at: http://localhost:5000/#/experiments/1/runs/0641a76c94f24dc39c80ce70e85af828
🧪 View experiment at: http://localhost:5000/#/experiments/1
```

Now, check the MLflow UI at `http://localhost:5000`. You should see a new experiment named "Lab1_Connectivity_Test" with a run called "Initial_CSV_Load".

Inside that run, you will find the logged parameters, metrics, and the dataset artifact, confirming that your connection to MLflow is successful and that your metadata is being tracked correctly!

Take your time to explore the MLflow UI. Click on the run to see the details, check the parameters and metrics, and even download the logged dataset to verify that it matches your original CSV. This is the first step in building a robust MLOps pipeline where every experiment is fully traceable and reproducible 😊

7. Summary of Actions & Branches

This structure allows us to evolve the project linearly across the course:

- **Branch `Lab1` (Current)**: You have successfully set up the structure, the Docker preliminary orchestration with MLflow, and tracked your first metadata.
- **Branch `Lab2`** : We will introduce **DVC** to manage the `data/` folder with content-based versioning.
- **Branch `Lab3`** : We will add **Hydra** to manage hyperparameter configurations dynamically.
- **Branch `Lab4`** : We will deploy **Airflow** to orchestrate these tasks into a daily weather update pipeline.